

PROJECT PROPOSAL

RideAway

Smart Ride Safety Application

ML-Powered Accident Detection & Emergency Response System for Riders

Document Type	Project Proposal
Project Name	RideAway – Smart Ride Safety
Platform	Flutter (Android / iOS)
Version	1.1 (ML Update)
Date	March 2026
Authors	L.G.S.B. Liyanage A.J.M.R.L. Jayasundara K.R.A.R. Jayathilaka M.S.Hafsa
Status	Under Development

1. Executive Summary

RideAway is a cross-platform mobile application built with Flutter that addresses a critical safety gap in the personal mobility sector. At its core is a trained Machine Learning model — running entirely on the device — that analyses real-time accelerometer and gyroscope sensor streams to intelligently classify crash events and automatically trigger emergency alerts to pre-configured contacts, accompanied by the rider's live GPS coordinates.

Road accidents involving motorcyclists, cyclists, and other personal vehicle operators represent a disproportionate share of traffic fatalities globally. In many incidents, the crucial window for emergency medical intervention is lost simply because no one is aware that an accident has occurred. RideAway eliminates this gap by acting as an always-on, intelligent safety guardian — one that understands the difference between a genuine crash and the ordinary vibrations of daily riding.

The defining innovation of RideAway over rule-based safety apps is its ML-driven detection engine. Fixed threshold approaches generate too many false alarms (speed bumps, hard braking, rough roads) or miss genuine crashes by failing to account for the multi-axis, time-series complexity of real impact signatures. RideAway's classification model — trained on labelled IMU sensor data — learns these patterns and generalises across device placements, rider types, and vehicle categories, achieving measurably lower false positive and false negative rates than threshold-based alternatives.

The application is built on a Flutter/Firebase stack. Phase 1 (authentication, onboarding, user profiles, routing) is complete. The ML model pipeline — data collection, feature engineering, model training, TFLite conversion, and on-device integration — is the centrepiece of Phase 2, currently in planning. The full roadmap covers the ML engine, emergency contact management, ride history, automated SMS alerts, and Google Maps live tracking.

2. Problem Statement

2.1 The Safety Gap

Riders face a unique vulnerability: in the event of a crash, they are often incapacitated and unable to call for help. Emergency response is time-critical. The key challenge is not just detecting that a crash has happened — it is detecting it automatically, reliably, and without requiring action from the injured rider.

Existing solutions fall into two categories. Hardware-based solutions (dedicated GPS trackers, airbag vests) are expensive, require additional devices, and are used by a small minority. Software-based solutions use fixed sensor thresholds to detect impacts — but these produce unacceptably high false positive rates because everyday riding events (speed bumps, potholes, aggressive braking) generate sensor spikes that cross fixed thresholds. The result: false alarms that erode user trust, leading riders to disable the feature entirely.

A Machine Learning approach addresses this directly. By training a model on labelled time-series sensor data representing both crash events and non-crash events (normal riding, bumps, braking), the model learns the full multi-dimensional signature of a crash — not just a single-axis magnitude — enabling it to classify events with far greater precision than any fixed-threshold rule system.

2.2 Why ML Is the Right Approach

- Crash events are not defined by a single sensor value but by a complex, multi-axis, temporal pattern — a task ideally suited to learned classifiers.
- Fixed thresholds cannot adapt to different device placements (trouser pocket vs. handlebar mount), which dramatically affect sensor readings for the same event.
- ML models can be retrained as more labelled data becomes available, continuously improving accuracy post-deployment without requiring app updates to the detection logic.
- On-device inference (TensorFlow Lite) enables the model to operate fully offline with sub-100 ms inference latency, meeting the real-time requirements of safety-critical applications.

2.3 Target Users

- Motorcycle and scooter riders commuting daily in urban environments
- Long-distance touring motorcyclists riding in remote areas
- Cyclists, including recreational and commuter cyclists
- E-scooter and e-bike riders in micro-mobility ecosystems
- Families and employers who need assurance of rider safety

2.4 Impact

A 2023 WHO report estimates that road traffic accidents cause approximately 1.19 million deaths each year globally, with riders of two- and three-wheeled vehicles representing nearly 28% of all road traffic deaths. Faster emergency notification is one of the most impactful non-vehicular interventions available. An ML-powered detection system that is trusted by riders — because it produces fewer false alarms — will be left running, ensuring it is active when it is genuinely needed.

3. Proposed Solution

3.1 Solution Overview

RideAway transforms the rider's own smartphone into an intelligent, ML-powered crash detection and emergency notification device. No additional hardware is required. The system runs a background service that continuously feeds accelerometer and gyroscope data into an on-device TensorFlow Lite model, which classifies each incoming sensor window as 'normal riding' or 'crash event' with a confidence score.

3.2 ML Accident Detection Engine

3.2.1 Data Collection & Labelling

The ML pipeline begins with data collection. Sensor data (accelerometer X/Y/Z + gyroscope X/Y/Z at 50 Hz) is recorded across two labelled event classes: crash events (simulated drops, falls, and controlled impact scenarios) and non-crash events (speed bumps, hard braking, potholes, rough road surfaces). Data is collected across multiple device placements to ensure placement-invariant generalisation.

3.2.2 Feature Engineering

Raw sensor time series are segmented into overlapping sliding windows (default: 2-second windows, 50% overlap). Per-window features are extracted including: resultant acceleration magnitude, per-axis mean and standard deviation, peak G-force, rate of change of acceleration (jerk), gyroscope magnitude, and frequency-domain features (FFT energy in low, mid, and high bands). These features form the input vector to the classifier.

3.2.3 Model Architecture & Training

The classification model targets a lightweight architecture suitable for on-device inference. Candidate architectures include a 1D Convolutional Neural Network (1D-CNN) operating directly on the raw time-series window, and a traditional ensemble classifier (Random Forest or Gradient Boosting) operating on the extracted feature vector. The 1D-CNN approach is preferred for its ability to learn temporal patterns without manual feature engineering. Training is performed in Python using TensorFlow/Keras, with the trained model converted to TensorFlow Lite (.tflite) format for deployment.

3.2.4 On-Device Inference (TFLite)

The .tflite model file is bundled with the Flutter application as an asset. The `tflite_flutter` Dart package loads and runs the model on the device CPU. Inference runs on each new sensor window (every ~1 second with 50% overlap), producing a binary classification label and a confidence score. A crash is flagged only when the model output exceeds a configurable confidence threshold (default: 0.85) for two consecutive windows, providing temporal consistency filtering that further reduces false positives.

3.3 Full System Functionality

- **The background service continuously samples IMU data and feeds it to the TFLite inference engine during active ride sessions.:** Passive Background Monitoring
- **The on-device model classifies each sensor window in real time, outputting a crash probability score.:** ML-Based Crash Classification
- **Upon ML detection, a 30-second on-screen countdown gives a conscious rider the chance to dismiss a false alarm before emergency flow triggers.:** Confirmation Window
- **Automated SMS messages with the rider's name and live GPS coordinates are dispatched to all registered emergency contacts.:** Emergency Notification
- **A dedicated UI allows users to register, edit, and remove trusted contacts stored in Firestore.:** Emergency Contact Management
- **Every session is logged with timestamps, route, distance, and detected events — including ML confidence scores for each flagged event.:** Ride History
- **Google Maps integration provides real-time location tracking during active rides.:** Live Location Tracking

3.4 Key Differentiators

- ML-powered detection — not fixed thresholds — producing significantly lower false positive rates
- Fully on-device inference: no internet required for the detection engine, no data sent to cloud for classification
- Model is retrainable as new labelled data is collected, enabling continuous accuracy improvement post-deployment
- Cross-platform: single codebase for Android and iOS via Flutter
- Firebase backend ensures real-time data sync and reliability for all non-detection features

4. Technology Stack

5.1 Application Framework

RideAway is developed using Flutter, Google's open-source UI toolkit for building natively compiled applications from a single Dart codebase. Flutter enables deployment to both Android and iOS with full access to native device APIs — critical for real-time sensor access, background service operation, and TensorFlow Lite integration.

5.2 Machine Learning Stack

- **The model training environment. A 1D Convolutional Neural Network (or Random Forest baseline) is trained on labelled accelerometer/gyroscope time-series data using TensorFlow 2.x / Keras in a Python environment (Jupyter Notebook / Google Colab).**: TensorFlow / Keras (Python)
- **The trained Keras model is converted to .tflite format using the TFLite converter, producing a small, optimised binary for on-device deployment. The conversion process applies post-training quantisation (int8) to reduce model size and inference latency.**: TensorFlow Lite
- **The Flutter package that loads the .tflite model asset, manages the interpreter lifecycle, and runs inference on input tensors constructed from the sliding-window sensor buffer.**: tflite_flutter (Dart package)
- **Used in the data processing and feature engineering pipeline for training data preparation, feature extraction, model evaluation (confusion matrix, precision, recall, F1), and cross-validation.**: NumPy / Pandas / Scikit-learn (Python)
- **Planned for over-the-air model updates: updated .tflite files can be pushed to Firebase ML without requiring an app store update, enabling continuous model improvement post-deployment.**: Firebase ML Model Hosting (Optional)

5.3 Backend Services (Firebase)

- **Manages user identity with support for email/password and Google OAuth.**: Firebase Authentication
- **Stores user profiles, emergency contacts, ride history, and per-session ML event logs (confidence score, GPS coordinates, outcome).**: Cloud Firestore
- **Push notification delivery to emergency contacts' devices.**: Firebase Cloud Messaging (FCM)
- **Stores the active ML model version identifier and confidence threshold defaults, allowing tuning without app updates.**: Firebase Remote Config

5.4 Key Flutter Packages

Package	Category	Purpose
<code>firebase_auth</code>	Authentication	User identity management
<code>cloud_firestore</code>	Database	NoSQL cloud data storage
<code>google_sign_in</code>	Authentication	Google OAuth provider
<code>tflite_flutter</code>	ML / AI	On-device TFLite model loading and inference
<code>sensors_plus</code>	Hardware	Accelerometer & gyroscope raw data access at 50 Hz
<code>geolocator</code>	Location	GPS coordinate streaming
<code>google_maps_flutter</code>	Mapping	Embedded map & route display
<code>flutter_background_service</code>	System	Persistent foreground service for continuous sensor + ML loop
<code>telephony / flutter_sms</code>	Communication	Automated SMS dispatch
<code>flutter_local_notifications</code>	Alerts	On-device alarm & notification delivery
<code>go_router</code>	Navigation	Declarative routing system
<code>firebase_remote_config</code>	ML / Config	OTA model version & threshold configuration

6. Project Plan & Timeline

6.1 Development Phases

Phase	Timeline	Deliverables	Milestone
Phase 1 Foundation	Weeks 1–3 (Complete)	Auth system, onboarding, Firestore user profiles, routing architecture	Working login & registration; first Firebase deployment
Phase 2 ML Pipeline	Weeks 4–7	Sensor data collection tool; labelled dataset assembly (crash / non-crash); feature engineering; model training (1D-CNN + RF baseline); TFLite conversion; on-device inference integration via tflite_flutter	ML model running on physical device; confusion matrix meeting accuracy targets
Phase 3 Safety Core	Weeks 8–10	Background sensor service feeding ML inference loop; confirmation dialog; emergency contacts UI; ML event logging to Firestore	End-to-end: ML detects crash → confirmation window appears → event logged
Phase 4 Notification	Weeks 11–12	SMS alert dispatch, FCM push notifications, GPS coordinate embedding in alerts	Successful SMS delivery to contact after confirmed ML detection
Phase 5 Mapping	Weeks 13–14	Google Maps live tracking, route polyline recording, ride history screen with ML event annotations	Full ride session logged with route replay and ML event markers
Phase 6 Polish & QA	Weeks 15–16	UI polish, detection threshold tuning, battery benchmarking, ML model OTA update pipeline via Firebase Remote Config	Beta release; false positive rate measured on real-device test suite
Phase 7 Launch	Weeks 17–18	Google Play & App Store submission, production Firebase rules, ML model v1.0 bundled	Public v1.0 release

6.2 Current Status

Phase 1 is complete. Authentication, Firebase integration, Firestore user profiles, and routing are implemented. Phase 2 (ML Pipeline) is next: the immediate focus is assembling the labelled training dataset and establishing the Python model training environment before proceeding to Flutter integration.

7. Risk Analysis

Risk	Likelihood	Impact	Mitigation
Insufficient labelled training data leading to poor ML model accuracy	Medium	Critical	Augment dataset with publicly available IMU crash datasets (SisFall, MobiFall); use data augmentation (time warping, jitter injection); collect additional controlled crash simulations
ML model false positive rate too high (normal riding events misclassified as crashes)	Medium	High	Multi-class training including hard negatives (speed bumps, potholes, hard braking); consecutive-window confirmation logic; user-tunable confidence threshold; 30-second dismissal window
ML model false negative rate too high (genuine crashes missed)	Low	Critical	Conservative default confidence threshold (0.85); both crash classes (high-speed vs. low-speed) represented in training data; ensemble of 1D-CNN + feature-based classifier
tflite_flutter inference latency exceeds real-time requirements on low-end devices	Medium	High	Post-training quantisation (int8) reduces model size and latency; benchmark on minimum-spec device (Android 8, 2 GB RAM); limit model input window to 2 seconds
Background service killed by OS during active ride	Medium	High	Foreground service with persistent notification; wake lock; battery optimisation exemption; auto-restart on kill via SERVICE_STICKY
ML model generalisation fails across device placements (pocket vs. handlebar)	Medium	Medium	Collect training data across multiple placements; include placement as a training variable; offer in-app placement calibration mode that applies a learned placement correction
SMS delivery failure (no signal in remote area)	Medium	High	Queue-and-retry on connectivity restore; FCM push as fallback; last known coordinates cached locally
Battery drain from continuous 50 Hz sensor + ML inference loop	Medium	Medium	Duty-cycle sampling; adaptive rate (reduce Hz during low-motion periods); ML battery benchmarking target < 5% per hour; profile with Android Battery Historian
Firebase ML OTA model update delivers a regressed model	Low	High	Staged rollout via Remote Config (5% → 25% → 100%); version pinning; fallback to bundled model if remote model fails checksum validation

8. Conclusion

RideAway represents a significant step forward from rule-based rider safety apps by replacing fixed sensor thresholds with a trained Machine Learning model. This architectural choice is not cosmetic — it directly addresses the primary reason existing safety apps fail in practice: excessive false alarms that erode user trust and lead to the app being disabled. By learning the true multi-dimensional, temporal signature of a crash event from labelled data, the ML engine generalises to the real-world diversity of device placements, vehicle types, and road conditions that no fixed rule system can adequately cover.

The technical stack is well-chosen for the task. TensorFlow Lite delivers the on-device inference performance and model size constraints required for a safety-critical background service. The Flutter/Firebase combination provides a mature, cross-platform foundation for all non-ML features. The ML pipeline — Python training environment through to `tflite_flutter` on-device integration — is an established, well-documented path with active community support.

With Phase 1 complete and the ML pipeline as the next major deliverable, RideAway is on track to deliver a working accident detection engine running on a physical device by Week 7. The subsequent phases build the emergency response, notification, and mapping features around that proven detection core. The result will be a production-ready safety application whose core claim — intelligent, reliable crash detection — is backed by a measurable, testable ML model rather than a set of arbitrary numbers.